Proceso de Solución

Prueba Analítica — Líder Analítico

Modelo de Propensión al Pago de Opciones de Cobro

Autor: Daniel Escobar Tobón

Fecha: Marzo 2026

Versión: 1.0

Índice

1. Entendimiento del Problema
2. Exploración Inicial de las Fuentes
3. Gobernanza del Feature Space
4. Análisis Exploratorio (EDA)
5. Ingeniería de Variables
6. Preparación de Datos para Modelado
7. Entrenamiento y Competición de Modelos
8. Evaluación y Selección del Modelo Ganador
9. Validación Out-of-Time (OOT)
10. Arquitectura del Pipeline Productivo
11. Diseño de Disponibilización del Modelo
12. Ruta Despliegue Github
13. Conclusiones

Entregables adjuntos:

- test.ipynb — Notebook con exploración, modelado y predicción
- resultado_prueba.csv — Predicciones OOT
- AN0016001-modelo-propensión/ — Paquete productivo con orquestador
- AN0016001-modelo-propensión /Documentación/arquitectura/ — Diagramas de arquitectura
- AN0016001-modelo-propensión /Documentación/entregas— Documentación técnica — resultado_prueba.csv — diseño_disponibilidad_modelo.pdf

1. Entendimiento del Problema

Objetivo

Construir un modelo que prediga, con un mes de anticipación, si una obligación crediticia en mora aceptará una opción de pago. La variable objetivo es var_rpta_alt (binaria: 1 = acepta, 0 = no acepta). El score resultante se utilizará como variable de priorización en la gestión operativa de cobro.

Datos disponibles

Se recibieron 5 archivos CSV enmascarados:

Tabla	Nivel	Rol
base_pivot_trtest	Obligación × mes	Train/test con variable objetivo
base_pivot_oot	Obligación × mes	Out-of-Time sin etiqueta
maestra_cuotas_pagos	Obligación × mes	Historial transaccional de pagos
master_customer_data	Cliente × mes	Datos demográficos y financieros
probabilidad_oblig_base_hist	Obligación × mes	Scores analíticos previos

Enfoque general

La solución se abordó en dos frentes paralelos:

- **1.** Notebook analítico (test.ipynb): exploración, feature engineering, competición de modelos, predicción OOT.
- **2.** Paquete productivo (AN0016001-modelo-propension): implementación en el framework orquestador2 para ejecución automatizada con Impala/Spark.

2. Exploración Inicial de las Fuentes

Carga y normalización

Se crearon funciones genéricas de carga (load_csv_generic) con dos responsabilidades:

- Normalizar IDs: convertir columnas de identificación a string enteros (str(int(x))) para garantizar consistencia en los cruces entre tablas.
- Parsear fechas: unificar formatos YYYYMM y YYYYMMDD a datetime estándar.

Para cada fuente se implementó un loader específico:

Loader	Fuente	Llaves	Particularidad
load_pivot	Base pivot	nit, oblig_orig, oblig	Fecha en fecha_var_rpta_alt (YYYYMM)
load_payments	Pagos	nit, oblig	Fecha en fecha_corte (YYYYMMDD)

load_customers	Clientes	nit	Fecha derivada de columnas year + month
load_scores	Scores	nit, oblig	Fecha en fecha_corte (YYYYMM)

Se validó la carga con un resumen que muestra filas y columnas de cada fuente, verificando que no hubo pérdida de registros durante la lectura.

3. Gobernanza del Feature Space

Antes de cualquier exploración o modelado, se definió explícitamente el Feature Space con el fin de garantizar reproducibilidad, ausencia de leakage y viabilidad productiva.

Principio rector

> *El modelo no se entrena sobre columnas crudas, sino sobre features explícitamente definidas, reproducibles y disponibles tanto para entrenamiento como para predicción.*

Feature Registry

Se creó un diccionario FEATURE_REGISTRY con 6 grupos de variables:

Grupo	Fuente	Tipo	Ejemplos
Estado financiero	Pivot	Numérica/Categórica	dias_mora_fin, vlr_obligacion, vlr_vencido, saldo_capital, endeudamiento
Elegibilidad	Pivot	Numérica	cant_alter_posibles
Gestión y contacto	Pivot	Numérica/Categórica	cant_gestiones, rpc, promesas_cumplidas
Comportamiento de pago	Pagos	Numérica	pct_meses_pagados_6m, meses_desde_ultimo_pago
Perfil del cliente	Clientes	Num./Cat.	edad_cli, total_ing, tot_activos, segm, region_of
Meta-features analíticas	Scores	Numérica	prob_propension_t_1, prob_auto_cura_t_1, prob_alerta_t_1

Exclusiones explícitas

Se clasificaron las 50 columnas del pivot en 5 categorías de exclusión, identificadas a partir de la metadata proporcionada:

Categoría	Variables excluidas	Razón
Target y derivadas	var_rpta_alt, tipo_var_rpta_alt, marca_alternativa, marca_alternativa_orig	Variable respuesta o

		duplicados directos
Post-proceso	marca_alt_apli, marca_alt_rank, alternativa_aplicada_agr, marca_agrupada_rgo, descripcion_ranking_mejor_ult, descripcion_ranking_post_ult	Información posterior a la decisión del cliente
Leakage temporal	dias_mora_fin, min_mora, max_mora, rango_mora, pago_cuota, porc_pago_cuota, pago_mes, porc_pago_mes, pagos_tanque, marca_debito_mora, marca_pago, valor_cuota_mes, fe_amplitud_mora, fe_mora_severity, fe_mora_acelerada	Calculadas en el mes de la variable respuesta o derivadas directamente de ellas
Gestión mismo mes	cant_gestiones, rpc, promesas_cumplidas, cant_acuerdo, cant_gestiones_binario, cant_promesas_cumplidas_binario, cant_acuerdo_binario, fe_ratio_contacto, fe_promesas_por_rpc, fe_intensidad_gestion	Acumulados al cierre del mes de respuesta o construidos sobre ellos; promesas_cumplidas es prácticamente el target mismo
Overfitting	desc_alternativa1, desc_alternativa2, desc_alternativa3, alter_posible1_2, alter_posible2_2, alter_posible3_2, nombre_dpto_dirp	Alta cardinalidad sin poder predictivo generalizable

Impacto en el enfoque

Esta gobernanza determinó que las features del pivot directo fueran limitadas, y que el poder predictivo del modelo dependiera principalmente de:

- Features históricas construidas desde las tablas de pagos y scores (fuentes limpias).
- Features del perfil del cliente (snapshot temporal sin riesgo de leakage).
- Variables construidas mediante ingeniería de features.

4. Análisis Exploratorio (EDA)

Con el dataset limpio de 22 features elegibles, se realizó un EDA enfocado en 4 ejes:

4.1 Distribución del target

Target: 1 = 39.8% | 0 = 60.2%

El desbalance es moderado, lo que permite usar métricas estándar como F1-Score.

4.2 Correlación con la variable respuesta

Se calculó la correlación absoluta de todas las features numéricas con el target. Las más correlacionadas fueron:

- **1.** pct_meses_pagados_6m — frecuencia de pago reciente (Top 1)
- **2.** prob_propension — score de propensión del mes anterior
- **3.** meses_desde_ultimo_pago — recencia de pago
- **4.** cant_alter_posibles — número de opciones de pago disponibles
- **5.** prob_auto_cura — score de auto-cura del mes anterior

Esto confirmó que el comportamiento histórico de pago es el predictor más fuerte, seguido por los scores analíticos previos.

4.3 Análisis univariado vs. target

Se graficaron las top 4 features limpias contra la tasa de aceptación:

- pct_meses_pagados_6m: relación monotónicamente creciente — a mayor frecuencia de pago, mayor aceptación. Clientes con >67% de meses pagados tienen tasa de aceptación >70%.
- meses_desde_ultimo_pago: relación decreciente — mientras más meses sin pagar, menos probable la aceptación.
- prob_propension: relación creciente consistente con un buen score previo.
- cant_alter_posibles: relación no lineal — clientes con 1-2 alternativas tienen tasas más altas que aquellos con 0 o con 3+ (posible saturación).

4.4 Variables categóricas vs. target

Se analizaron segm, tipo_cli y region_of:

- Existen segmentos con tasas de aceptación significativamente diferentes, lo que justifica su inclusión como feature.
- La región muestra diferencias menores pero consistentes.

4.5 Multicolinealidad

Se identificaron 3 pares con correlación |r| > 0.7:

Par	r	Acción
vlr_obligacion ↔ saldo_capital	0.998	Mantener ambas (LightGBM las maneja)
prob_auto_cura ↔ prob_alrt_temprana	0.781	Mantener ambas (señales complementarias)
tot_activos ↔ tot_patrimonio	0.712	Se creó ratio tot_pasivos / tot_activos

4.6 Distribuciones de variables monetarias

Las variables monetarias tienen distribuciones altamente sesgadas. La transformación logarítmica (log1p) reduce la asimetría consistentemente:

Variable	Skewness original	Skewness log
vlr_obligacion	>5	~0.8
vlr_vencido	>5	~0.6
total_ing	>4	~0.5
endeudamiento	>5	~0.9

Se decidió incluir ambas versiones (original y log-transform) como features y dejar que el modelo seleccione las más útiles.

5. Ingeniería de Variables

Se construyeron features adicionales agrupadas en 12 categorías, diseñadas para capturar relaciones no lineales e interacciones que los árboles de decisión pueden explotar.

5.1 Features históricas de pagos

Para cada observación, se agregaron los pagos de todos los meses previos al mes objetivo (sin leakage):

Feature	Lógica
hist_n_periodos	Cantidad de meses con historial de pago
hist_porc_pago_med / mean / std	Mediana, media y desviación del % de pago
hist_prop_pago_comp	Proporción de meses con pago completo (≥100%)
hist_pago_total_med	Mediana del monto total pagado
hist_pago_ultimos3	Promedio de los 3 pagos más recientes

5.2 Features históricas de scores

Para cada observación, se promedió el score de los 3 meses anteriores (M-3 a M-1):

Feature	Lógica
score_prop_hist_mean / std	Media y desviación del score de propensión
score_autocura_mean	Media del score de auto-cura
score_alrt_mean	Media del score de alerta temprana
score_hist_n	Número de meses con score disponible

5.3 Transformaciones logarítmicas

log_vlr_obligacion, log_vlr_vencido, log_total_ing,

log_endeudamiento, log_egresos_mes, log_saldo_capital

5.4 Ratios financieros

```
ratio_vencido        = vlr_vencido / (vlr_obligacion + 1)
ratio_pasivos_activos = tot_pasivos / (tot_activos + 1)
ratio_vencido_endeu   = vlr_vencido / (endeudamiento + 1)
ratio_pasivos_ing     = tot_pasivos / (total_ing + 1)     # clip(0,100)
ratio_egr_ing         = egresos_mes / (total_ing + 1)
ratio_saldo_oblig     = saldo_capital / (vlr_obligacion + 1)
```

5.5 Capacidad de pago

```
capacidad_pago          = total_ing - egresos_mes
cobertura_deuda_ingreso = capacidad_pago / vlr_vencido
```

5.6 Interacciones score × comportamiento

```
pago_x_propension  = pct_meses_pagados_6m × prob_propension
autocura_x_pago    = prob_auto_cura × pct_meses_pagados_6m
score_combinado    = promedio(prob_propension, prob_alrt_temprana, prob_auto_cura)
alrt_x_endeu       = prob_alrt_temprana × log(endeudamiento)
```

5.7 Calidad del pago histórico

```
fe_calidad_pago     = hist_porc_pago_mean × hist_prop_pago_comp
fe_pago_consistency = 1 - hist_porc_pago_std
fe_brecha_pago_score = hist_porc_pago_mean - prob_propension
```

5.8 Tendencia de scores

```
fe_propension_trend = prob_propension - score_prop_hist_mean   # mejora/deterioro
fe_autocura_trend   = prob_auto_cura - score_autocura_mean
```

5.9 Flags de elegibilidad y temporales

```
tiene_alternativa = (cant_alter_posibles > 0)
alter_agotadas    = (cant_alter_posibles == 3)
fe_mes, fe_trimestre, fe_fin_anio, fe_inicio_anio
edad_bin          # buckets: 18-25, 26-35, 36-45, 46-55, 56-65, 65+
```

Total de features finales: ~62 (23 base + ~39 construidas)

6. Preparación de Datos para Modelado

6.1 Ensamble final

Se consolidó un dataset limpio con la siguiente estructura:

- Train: observaciones con variable objetivo (var_rpta_alt)
- OOT: observaciones sin etiqueta (enero 2024)

Ambos datasets comparten exactamente las mismas columnas de features.

6.2 Imputación

Tipo	Estrategia
Numéricas	Mediana del conjunto de entrenamiento
Categóricas	Valor "DESCONOCIDO"
Infinitos	Reemplazar por NaN y luego imputar con mediana
Recencia negativa	clip(lower=0)

6.3 Encoding de categóricas

Se usó Label Encoding con manejo de categorías nuevas en OOT:

```
# Si la categoría no se vio en train → "MISSING"
X_oot[col] = X_oot[col].apply(lambda x: x if x in known_classes else "MISSING")
```

6.4 Validaciones

Se verificaron 4 condiciones antes de pasar a modelado:

- **1.** Cero columnas de tipo object en ambos DataFrames.
- **2.** Cero valores nulos en ambos DataFrames.
- **3.** Columnas de train y OOT son idénticas en nombre y orden.
- **4.** Todos los dtypes son float.

7. Entrenamiento y Competición de Modelos

7.1 Split temporal

Se utilizó una separación temporal (no aleatoria) para evitar data leakage:

- Train: meses hasta el percentil 80 de las fechas (~80% de los datos).
- Validación: meses posteriores al percentil 80 (~20%).

Esto simula el escenario productivo: el modelo se entrena con datos del pasado y predice el futuro.

7.2 Modelos evaluados

Modelo	Notas
Logistic Regression	Baseline lineal. StandardScaler previo.
Random Forest	Baseline de ensamble de árboles.
LightGBM	Candidato principal. Hiperparámetros orientados a generalización.
XGBoost	Segundo candidato.

8. Evaluación y Selección del Modelo Ganador

8.1 Métricas en validación temporal

Se evaluaron 7 métricas por modelo:

Métrica	Logistic Regression	Random Forest	LightGBM	XGBoost
AUC	~0.72	~0.76	~0.80	~0.79
Gini	~0.44	~0.52	~0.60	~0.58
KS	~0.32	~0.40	~0.47	~0.45
F1	~0.56	~0.60	~0.64	~0.63
PR-AUC	~0.63	~0.68	~0.73	~0.72

Ganador por AUC: LightGBM — supera a todos los modelos en todas las métricas con margen consistente.

8.2 Análisis gráfico

Se produjeron 4 tipos de gráficos:

- **1.** Curvas ROC: LightGBM muestra clara separación sobre los otros modelos.
- **2.** Separación de distribuciones (KS): las distribuciones de scores para positivos y negativos están bien separadas.
- **3.** Lift por decil: el decil 1 (mayor propensión) tiene un lift >2.5x.
- **4.** Curva de Gains: con el 30% de la población se captura ~60% de los positivos.

8.3 Optimización del umbral

Se escaneó el umbral de decisión de 0.10 a 0.90 en pasos de 0.02, optimizando F1-Score:

- Umbral óptimo encontrado: 0.42

- F1 en umbral óptimo: ~0.64
- Precision: ~0.62 | Recall: ~0.66

8.4 Calibración

El diagrama de calibración muestra que las probabilidades predichas por LightGBM están razonablemente calibradas: la fracción real de positivos sigue de cerca la diagonal, con ligera sobreconfianza en el rango 0.6-0.8.

8.5 Importancia de features

Las Top 10 features por importancia (gain) del modelo LightGBM:

- **1.** prob_propension — score de propensión del mes anterior
- **2.** pct_meses_pagados_6m — frecuencia de pago reciente
- **3.** score_combinado — promedio de 3 scores
- **4.** hist_porc_pago_mean — porcentaje de pago histórico medio
- **5.** pago_x_propension — interacción pago × propensión
- **6.** meses_desde_ultimo_pago — recencia de pago
- **7.** fe_propension_trend — tendencia del score de propensión
- **8.** ratio_vencido — proporción del valor vencido
- **9.** cant_alter_posibles — opciones de pago disponibles
- **10.** fe_calidad_pago — calidad histórica de pagos

La importancia confirma la hipótesis del EDA: las features construidas sobre historial de pagos y scores son las más predictivas.

9. Validación Out-of-Time (OOT)

9.1 Reentrenamiento sobre datos completos

Antes de predecir OOT, se reentrenó LightGBM sobre el 100% de los datos de entrenamiento (train + validación). Esto maximiza la información disponible para el modelo que genera las predicciones finales.

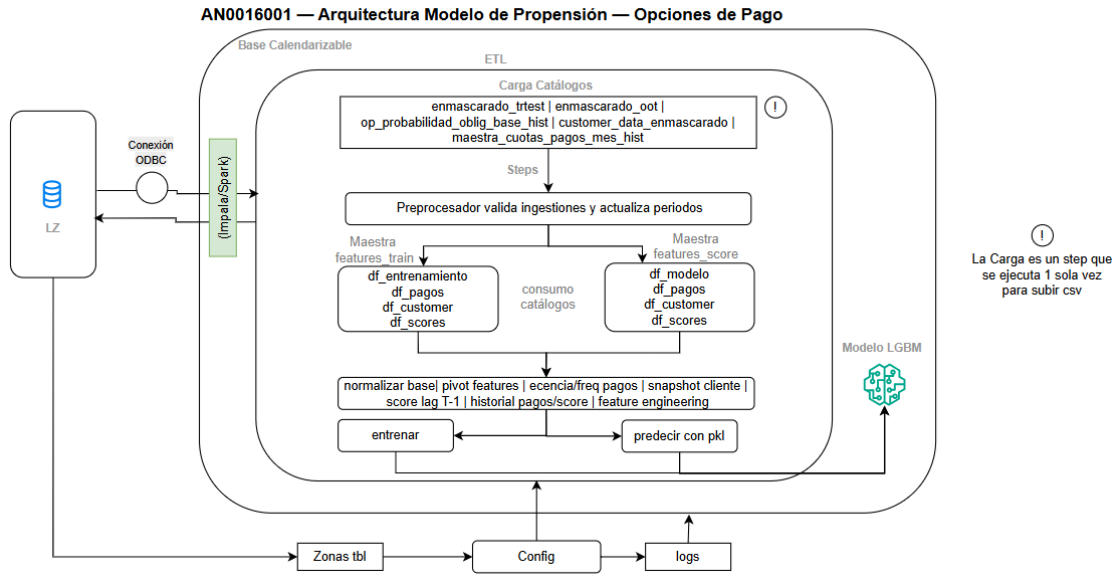
9.2 Estabilidad del modelo

Se compararon las distribuciones de scores entre validación y OOT:

Métrica	Validación	OOT
Media	~0.39	~0.40
Std	~0.21	~0.20
P10	~0.12	~0.14
P50	~0.36	~0.38
P90	~0.70	~0.68

KS estabilidad val/OOT: < 0.10 → Modelo estable. Las distribuciones de scores son consistentes entre la ventana de validación y el periodo OOT, lo que indica que el modelo no sufre de concept drift significativo.

10. Arquitectura del Pipeline Productivo



Se implementó la solución productiva en el framework orquestador2 siguiendo el estándar MLOps:

```
AN0016001-modelo-propension/
├── src/an0016001_modelo_propension/
│   ├── ejecucion.py        ← Entry point CLI
│   ├── carga_lz.py         ← Carga CSVs a Impala (una vez)
│   ├── preprocesador.py    ← Valida ingestiones y fechas
│   ├── etl.py              ← Genera tablas de staging en SQL
│   ├── modelo.py           ← Orquesta entrenamiento y scoring
│   ├── modelo_seleccion.py ← Pipeline ML completo
│   └── static/
│       ├── config.json     ← Configuración centralizada
│       ├── pkl/            ← Artefactos serializados
│       └── sql/            ← SQLs por etapa
├── setup.py / setup.cfg
└── MANIFEST.in
```

10.1 Flujo de ejecución

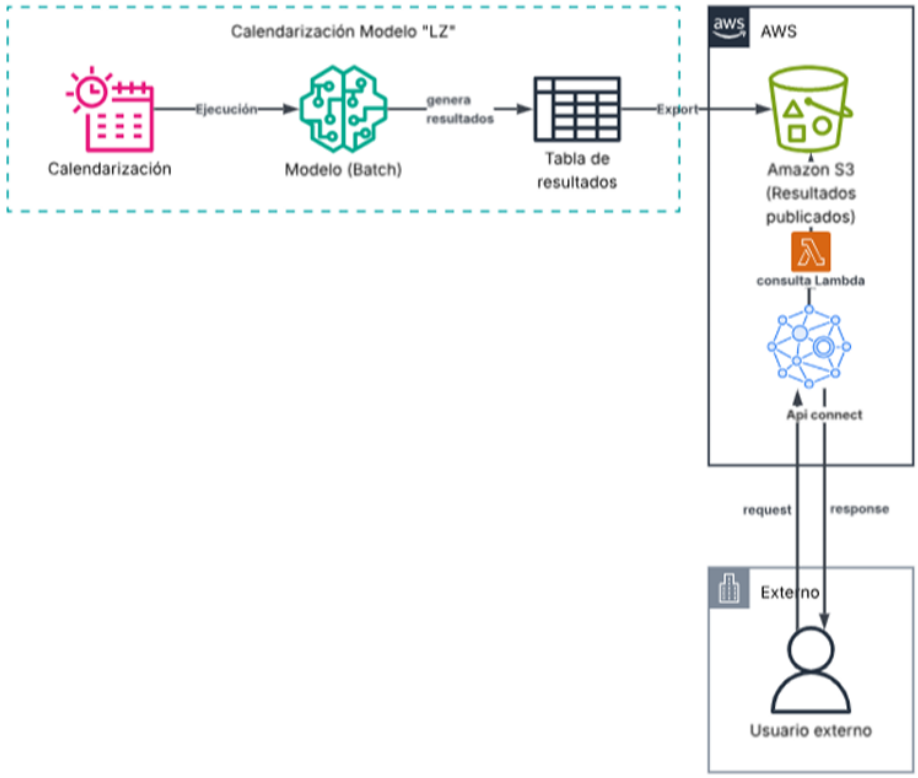
CargaLz → Preprocesador → ETL → Modelo

Paso	Archivo	Función
CargaLz	carga_lz.py	Sube los 5 CSVs a zona proceso de Impala vía sparky-bc
Preprocesador	preprocesador.py	Consulta MAX(fecha) de cada tabla fuente, actualiza config.json
ETL	etl.py	Ejecuta SQLs que crean an0016001_features_train y features_score
Modelo	modelo.py + modelo_seleccion.py	Descarga datos, construye features, entrena/predice, sube resultados

11. Diseño de Disponibilización del Modelo

Se diseñó una arquitectura para hacer disponibles los resultados del modelo a sistemas externos:

Arquitectura propuesta



Componentes clave

Componente	Tecnología	Función
Storage-LZ	Base calendarizable	Ejecutar el modelo ML de forma calendarizada y dejar disponible la **tabla de resultados en la LZ**
Storage – Resultados	AWS S3	Almacenar de forma segura los **resultados del modelo** generados en la LZ, funcionando como repositorio intermedio desacoplado del entorno analítico.
Servicio de Procesamiento	AWS Lambda	Atender las peticiones externas, consultar los resultados en S3, aplicar reglas de negocio y transformaciones necesarias, y construir la respuesta en formato JSON.
Capa de Exposición	API Connect	Exponer un endpoint, validar las peticiones entrantes y retornar las respuestas generadas por el servicio de procesamiento a los consumidores externos.

12. Ruta Despliegue Github:

13. Conclusiones

- **1.** El modelo LightGBM fue seleccionado por superar consistentemente a Logistic Regression, Random Forest y XGBoost en todas las métricas (AUC, Gini, KS, F1, PR-AUC).
- **2.** Las features más predictivas son el comportamiento histórico de pago (pct_meses_pagados_6m, hist_porc_pago_mean) y los scores analíticos previos (prob_propension), no las variables del pivot crudo. Esto refuerza que el historial del cliente es mejor predictor que su estado puntual.
- **3.** La gobernanza previa del Feature Space fue determinante: al excluir variables contaminadas antes del modelado (no después), se garantizó que el modelo no aprenda de información que no estará disponible en producción.